Before the Search Begins: Generating Function-Informed Initialization for Tight Constrained Binary Optimization

Muhammad Marzouk Baig
MS Candidate, Khoury College of Computer Sciences
Northeastern University, Portland, Maine
baig.muham@northeastern.edu

Abstract

We present a generating function-informed initialization framework for genetic algorithms (GAs) applied to the Multidimensional Knapsack Problem (MKP). Standard initialization—setting each bit to 1 with probability 0.5 independently—ignores the feasibility structure of the search space entirely. On tight instances this produces initial populations with zero feasible solutions, preventing any search from proceeding under constraint-respecting fitness models. The framework is grounded in the combinatorial generating function tradition, applied here to constraint feasibility rather than partition enumeration: for each item j, we derive a sampling probability p_j from sub-generating functions that measure how much feasible space is lost when item j is included versus excluded. The framework is purely constraint-driven: it requires no objective function, no LP relaxation, and no prior search. A GPU-accelerated Monte Carlo approximation provides estimates accurate to within 0.03% in 5–8 seconds. On tight instances ($\alpha = 0.25$), where uniform initialization produces zero feasible solutions, our method achieves 52.5% initial feasibility and converges to 96.5% of the reference best under a death penalty model—a regime where the uniformly initialized GA fails entirely. Adding a lightweight iterated local search post-processing step raises mean quality to 99.3%. The framework is designed for tight-to-medium instances ($\alpha \leq 0.50$) where feasibility is genuinely scarce; on loose instances it degenerates automatically to uniform initialization.

1 Introduction

1.1 Motivation

Population initialization is the first decision a genetic algorithm makes, yet it is almost universally treated as a formality. The standard approach—sampling each bit independently with probability 0.5—is convenient and assumption-free, but it is provably oblivious to the structure of the problem being solved. For unconstrained or loosely constrained problems this is harmless. For tightly constrained combinatorial problems, it can be catastrophic.

The generating function connection. Generating functions are a classical tool in combinatorics for encoding subset enumeration in compact algebraic form. The idea goes back at least to Euler and reached remarkable depth in the work of Hardy and Ramanujan [7] on integer partitions: the generating function $\prod_{k=1}^{\infty}(1-x^k)^{-1}$ encodes, in the coefficient of x^n, the number of ways to write n as a sum of positive integers. The same algebraic principle applies to our setting. For items with integer weights $w_1, \ldots, w_n$, the product $\prod_j(1+x^{w_j})$ encodes, in the coefficient of x^k, the number of subsets with total weight exactly k. This maps naturally onto the feasibility question for constrained binary optimization: by factoring out one item at a time and evaluating the remainder against each capacity threshold, we can ask—before any search begins—how much of the feasible space survives when item j is included versus excluded. To our knowledge, this connection has not previously been exploited to bias GA initialization. The framework we develop shows that it yields a principled, interpretable, and provably self-calibrating initialization method.

The tight instance failure mode. Consider the MKP with a tightness ratio $\alpha = W_i / \sum_j w_{ji}$ of 0.25: each capacity is set to 25% of the total item weight across that constraint. On such instances the fraction of all binary strings that are feasible is tiny. Uniform random initialization samples from the full $\{0,1\}^n$ space without any awareness of where feasible solutions lie; in practice, every solution in an initial population of 50 is infeasible. Under a death penalty fitness model—which assigns zero fitness to infeasible solutions—the entire initial population is non-competing and selection cannot operate. The GA is dead on arrival. This failure mode is not an artifact of a poorly chosen penalty function or an implementation bug. It is a structural consequence of initializing without any knowledge of where feasible solutions live.

The scope of repair-based approaches. The most effective existing technique for handling infeasibility in MKP GAs is the deterministic repair operator of Chu and Beasley [1], which uses an efficiency ranking $u_j = v_j / \sum_i (w_{ji}/W_i)$ to greedily add or remove items until feasibility is restored. This operator is computationally efficient and empirically effective on standard benchmarks. However, it depends entirely on objective values: the ranking cannot be computed when objective values are absent (pure constraint satisfaction), unreliable (noisy or estimated objectives), or misleading (adversarial instance structure). In these settings, constraint-only initialization methods become the primary available tool for guiding early search.

1.2 Contributions

The specific contributions of this paper are:

1. A mathematical framework that uses subgenerating functions over order-1 schemas to derive per-item feasibility densities ρ_j^1 and ρ_j^0 for any binary constrained optimization instance (Section 3).

2. A sampling rule $p_j = \rho_j^1/(\rho_j^1 + \rho_j^0)$ that recovers uniform initialization when no constraint structure is present ($p_j = 0.5$) and biases away from infeasibility-inducing items in proportion to the feasibility cost they impose (Section 3).

3. A GPU-accelerated Monte Carlo approximation algorithm that estimates ρ_j^1 and ρ_j^0 to within 0.03% in 5–8 seconds for $n = 100$, $m = 3$, running in $O(n \cdot M \cdot m)$ time with $M = 10^7$ samples (Section 4).

4. Empirical results on MKP instances at three tightness levels ($\alpha \in \{0.25, 0.50, 0.75\}$) showing that GF-biased initialization enables search under a death penalty model in regimes where uniform initialization completely fails, while matching uniform performance on easier instances. We also show that the framework composes naturally with standard post-processing techniques (Section 7).

1.3 Paper Organization

Section 2 reviews the MKP, the Chu–Beasley GA, schema theory, and generating functions. Section 3 develops the mathematical framework. Section 4 presents the exact and Monte Carlo algorithms along with a complexity analysis. Section 5 provides worked examples on small instances. Section 6 describes the experimental setup. Section 7 presents results. Section 8 discusses limitations and the broader applicability of the framework. Section 9 concludes.

2 Background

2.1 The Multidimensional Knapsack Problem

The MKP generalizes the classical 0-1 knapsack problem to m constraints. Given n items each with value v_j and weight w_{ji} for constraint i, and m capacity constraints W_i, the MKP is:

$$\begin{aligned} &\text{maximize} \quad \sum_{j=1}^{n} v_j x_j \\ &\text{subject to} \quad \sum_{j=1}^{n} w_{ji} x_j \le W_i \quad \forall i = 1, \ldots, m \\ &x_j \in \{0, 1\} \quad \forall j = 1, \ldots, n \end{aligned}$$

The MKP is NP-hard and is a canonical benchmark for metaheuristics. Instances are commonly parameterized by the *tightness ratio* $\alpha = W_i / \sum_j w_{ji}$, which controls the fraction of the total item weight that fits in each constraint. Standard benchmark instances are available from the OR-Library [2].

2.2 Chu and Beasley Genetic Algorithm

Chu and Beasley [1] proposed a steady-state GA for the MKP that remains the standard baseline. Key components:

- **Initialization:** Each bit x_j set to 1 with probability 0.5 independently.
- **Crossover:** Uniform crossover.
- **Mutation:** Bit-flip at rate $1/n$.
- **Selection:** Binary tournament.
- **Repair:** A deterministic operator that iteratively drops items (in ascending order of efficiency u_j) until feasibility is restored, then greedily adds items (in descending order of u_j) while feasibility is maintained, where:

$$u_j = \frac{v_j}{\sum_{i=1}^{m} w_{ji}/W_i}$$

The repair operator is effective on standard benchmarks because item efficiency is a reliable proxy for solution quality when values and weights are uncorrelated. It is undefined when no objective exists and potentially misleading when value-weight correlation is adversarial [5]. Because the repair ranking is fixed, diverse infeasible offspring tend to be repaired toward similar feasible solutions, which can contribute to premature convergence on harder instances.

2.3 Schema Theory

Schema theory, introduced by Holland [3], provides a framework for analyzing GA behavior at the level of partial solutions. A *schema* is a string of length n over $\{0, 1, *\}$, where $*$ is a wildcard matching either value. An *order-k* schema has k fixed positions. The Schema Theorem states that short, low-order, above-average schemata are sampled exponentially more frequently across generations. Initialization determines the initial representation frequency of all schemata; biasing toward high-feasibility order-1 schemata directly seeds the population with the building blocks from which feasible high-quality solutions will be composed.

2.4 Generating Functions in Combinatorics

A generating function is a formal power series whose coefficients encode combinatorial quantities. The tradition is ancient: Euler used generating functions to analyze integer partitions, and the field reached a landmark in the Hardy–Ramanujan asymptotic formula [7] for the partition function $p(n)$, derived by analyzing $\prod_{k=1}^{\infty}(1-x^k)^{-1}$. For a finite set of items with integer weights $w_1, \ldots, w_n$, the analogous product is:

$$f(x) = \prod_{j=1}^{n}(1 + x^{w_j})$$

The coefficient of x^k in $f(x)$ equals the number of subsets with total weight exactly k. This follows from the distributive law: each factor contributes either 1 (item excluded) or x^{w_j} (item included), so the product enumerates all 2^n subsets. Wilf [4] provides a comprehensive treatment of generating functions in combinatorics; the specific application to constraint feasibility and GA initialization developed here is, to our knowledge, new.

3 Mathematical Framework

3.1 Feasibility and the Full Generating Function

Let there be n items and m constraints. A solution $\mathbf{x} \in \{0, 1\}^n$ is *feasible* if:

$$\sum_{j=1}^{n} w_{ji} x_j \leq W_i \quad \forall i = 1, \ldots, m$$

For m constraints, the full generating function over all 2^n subsets is:

$$f(x_1, \ldots, x_m) = \prod_{j=1}^{n} \left(1 + x_1^{w_{j1}} x_2^{w_{j2}} \cdots x_m^{w_{jm}}\right)$$

A term $x_1^{k_1} \cdots x_m^{k_m}$ in the expanded product corresponds to a feasible subset if and only if $k_i \leq W_i$ for all i.

3.2 Sub-Generating Functions and Feasibility Densities

To analyze feasibility at the schema level, we define the *sub-generating function* for item j as the generating function over all $n-1$ remaining items:

$$g_j(x_1, \ldots, x_m) = \prod_{k \neq j} \left(1 + x_1^{w_{k1}} \cdots x_m^{w_{km}}\right)$$

This function enumerates all 2^{n-1} subsets of items other than j. The function g_j is the same regardless of whether item j is fixed to 0 or 1; the distinction lies solely in the capacity threshold applied when counting feasible terms:

- When item j is **included** ($x_j = 1$), its weight consumes capacity, leaving $W_i - w_{ji}$ for the remaining items. The feasibility density is:

$$\rho_j^1 = \frac{\#\{\text{terms in } g_j : \text{exponent of } x_i \leq W_i - w_{ji}\ \forall i\}}{2^{n-1}}$$

- When item j is **excluded** ($x_j = 0$), full capacity is available. The feasibility density is:

$$\rho_j^0 = \frac{\#\{\text{terms in } g_j : \text{exponent of } x_i \leq W_i\ \forall i\}}{2^{n-1}}$$

Intuitively, ρ_j^1 is the probability that a uniformly random solution *containing* item j is feasible; ρ_j^0 is the same for solutions *not* containing j.

3.3 Order-1 Schemas and the Schema-Level Interpretation

A schema is a string over $\{0, 1, *\}$ where $*$ is a wildcard. An order-1 schema has exactly one fixed position. For n items there are $2n$ order-1 schemas—two per item—each containing exactly 2^{n-1} strings. The sub-generating function g_j corresponds exactly to the order-1 schema for item j: it enumerates and characterizes the feasibility of all 2^{n-1} strings that agree with item j's fixed value. Restricting the analysis to order-1 structures requires n sub-generating function computations (one per item) rather than the 3^n required for full schema enumeration.

3.4 Sampling Probability

We allocate the initialization probability for bit j proportionally to the feasibility signal of each choice. For two mutually exclusive options with scores a and b, the proportional allocation rule assigns probability $a/(a+b)$ to option a. Applied to our two feasibility densities:

$$p_j = \frac{\rho_j^1}{\rho_j^1 + \rho_j^0}$$

This rule satisfies two important properties:

1. $p_j + (1 - p_j) = 1$ (the probabilities sum to one by construction).

2. The ratio of probabilities equals the ratio of feasibility densities: if excluding item j is twice as feasibility-preserving as including it, the algorithm is twice as likely to exclude it.

Boundary cases. When $\rho_j^1 = \rho_j^0$, we have $p_j = 0.5$, recovering standard uniform initialization. This occurs on loose instances (α near 1), where the presence or absence of any single item has negligible effect on feasibility. When $\rho_j^1 = 0$, item j is never compatible with any completion of the remaining items—$p_j = 0$ and the item is never selected. When $\rho_j^0 = 0$, all completions are infeasible regardless of whether j is included or not, which cannot occur on any instance for which the empty solution is feasible.

4 Algorithm

4.1 Exact Computation

The exact procedure computes ρ_j^1 and ρ_j^0 by explicitly enumerating all 2^{n-1} terms of each sub-generating function. Note that the full generating function f itself is never expanded; only the n sub-generating functions g_j are required, each of size 2^{n-1}.

Algorithm 1 GF-Informed Initialization (Exact)

Require: n items, m constraints, weights w_{ji}, capacities W_i, population size P
Ensure: Initial population of P solutions
Phase 1 — Precomputation (done once)
for $j = 1$ to n **do**
  Expand $g_j(x_1, \ldots, x_m) = \prod_{k \neq j} (1 + x_1^{w_{k1}} \cdots x_m^{w_{km}})$
  $\rho_j^1 \leftarrow$ fraction of terms where exponent of $x_i \leq W_i - w_{ji}$ for all i
  $\rho_j^0 \leftarrow$ fraction of terms where exponent of $x_i \leq W_i$ for all i
  $p_j \leftarrow \rho_j^1 / (\rho_j^1 + \rho_j^0)$
end for
Phase 2 — Population Initialization
for $s = 1$ to P **do**
  for $j = 1$ to n **do**
    $x_j \leftarrow 1$ with probability p_j, else 0
  end for
  Add solution $\mathbf{x}$ to population
end for
return population

Exact computation requires enumerating 2^{n-1} terms per item and is feasible only for $n \leq 20$ in practice. Phase 2 is identical to standard uniform initialization except each bit j is sampled with probability p_j rather than a fixed 0.5.

4.2 Monte Carlo Approximation

For practical instances ($n = 50$–500), exact enumeration is intractable. We approximate ρ_j^1 and ρ_j^0 via Monte Carlo sampling. Instead of enumerating all 2^{n-1} completions, we draw M random binary vectors $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)}$ over the $n-1$ items other than j, and count the fraction that are feasible under each threshold:

$$\hat{\rho}_j^1 = \frac{1}{M} \sum_{k=1}^{M} \mathbf{1}\left[\sum_{\ell \neq j} w_{\ell i}\, s_\ell^{(k)} \leq W_i - w_{ji} \;\; \forall i\right]$$

$$\hat{\rho}_j^0 = \frac{1}{M} \sum_{k=1}^{M} \mathbf{1}\left[\sum_{\ell \neq j} w_{\ell i}\, s_\ell^{(k)} \leq W_i \;\; \forall i\right]$$

where $s_\ell^{(k)} \overset{\text{i.i.d.}}{\sim} \text{Bernoulli}(0.5)$.

Accuracy. Since each $\mathbf{1}[\cdot]$ is a Bernoulli random variable, the standard error of $\hat{\rho}_j^1$ is $\sqrt{\rho_j^1(1-\rho_j^1)/M} \leq 1/(2\sqrt{M})$. At $M = 10^7$, this upper bound is 0.016%—sufficient for initialization purposes.

GPU acceleration. The $M \times n$ matrix of random binary samples can be generated and evaluated entirely on a GPU using PyTorch tensor operations. On a T4 GPU (Google Colab), computing all n probability estimates for $n = 100$, $m = 3$, $M = 10^7$ takes 5–8 seconds. This is a one-time precomputation before the GA begins.

Failure mode. When $\hat{\rho}_j^1 = 0$ (no Monte Carlo sample is feasible with item j included), $p_j = 0$ and item j is never selected. This is the correct behavior when item j truly blocks all feasible completions, but can be an estimation artifact on very tight instances. In our experiments at $\alpha = 0.25$, some items have estimated $p_j = 0$ (never include) and others $p_j = 1$ (always include), reflecting genuine extreme constraint structure rather than sampling failure.

4.3 Complexity Analysis

Exact computation. Expanding each sub-generating function g_j enumerates 2^{n-1} terms, and checking feasibility against m constraints requires $O(m)$ work per term. Since there are n items, Phase 1 costs $O(n \cdot 2^n \cdot m)$—exponential in n and intractable for $n > 20$ in practice.

Monte Carlo approximation. Each of the n items requires M sample evaluations, each checking m constraints. Phase 1 therefore costs $O(n \cdot M \cdot m)$—linear in n, M, and m, and fully parallelizable across the M sample dimension on a GPU. Concretely, for $n = 100$, $m = 3$, $M = 10^7$, this involves approximately 3×10^9 scalar operations, parallelized to 5–8 seconds on a T4 GPU. The approximation thus reduces an exponential enumeration to a polynomial estimation at the cost of a controllable error bounded by $1/(2\sqrt{M})$.

Phase 2 (population initialization) costs $O(P \cdot n)$ in both cases and is dominated by Phase 1 for any reasonable population size P.

Comparison to LP-based approaches. LP relaxation-based initialization methods require solving an optimization subproblem and depend on an objective function. The GF Monte Carlo approach requires only weights and capacities, solves no optimization problem, and scales linearly in n, m, and M.

Memory. The Monte Carlo approach stores an $M \times (n-1)$ binary matrix in GPU memory. At $M = 10^7$ and $n = 100$, this is approximately 125 MB—manageable on modern hardware and straightforwardly batchable if memory is limited.

5 Worked Examples

5.1 Single Constraint Example

Consider the following MKP instance with $n = 3$ items and $m = 1$ constraint:

	Item 1	Item 2	Item 3
Weight	3	2	4
Value	5	4	3

Table 1. *3-item single constraint instance. Capacity $W = 5$.*

Step 1 — Full Generating Function

$$f(x) = (1 + x^3)(1 + x^2)(1 + x^4)$$
$$= 1 + x^2 + x^3 + x^4 + x^5 + x^6 + x^7 + x^9$$

Of the $2^3 = 8$ subsets, the feasible ones (exponent ≤ 5) are: $\emptyset$, $\{2\}$, $\{1\}$, $\{3\}$, $\{1, 2\}$—giving overall feasibility density $5/8 = 0.625$.

Step 2 — Sub-Generating Functions

Each $g_j(x)$ is the product over the two items other than j and is evaluated against two thresholds: $W - w_j$ (item included) and W (item excluded).

Item 1 ($w_1 = 3$, $W - w_1 = 2$): $g_1(x) = (1+x^2)(1+x^4) = 1 + x^2 + x^4 + x^6$. Of 4 terms: 2 have exponent ≤ 2 and 3 have exponent ≤ 5.

$$\rho_1^1 = 0.50 \quad \rho_1^0 = 0.75 \quad p_1 = 0.40$$

Item 2 ($w_2 = 2$, $W - w_2 = 3$): $g_2(x) = (1+x^3)(1+x^4) = 1 + x^3 + x^4 + x^7$. Of 4 terms: 2 have exponent ≤ 3 and 3 have exponent ≤ 5.

$$\rho_2^1 = 0.50 \quad \rho_2^0 = 0.75 \quad p_2 = 0.40$$

Item 3 ($w_3 = 4$, $W - w_3 = 1$): $g_3(x) = (1+x^3)(1+x^2) = 1 + x^2 + x^3 + x^5$. Of 4 terms: 1 has exponent ≤ 1 and 4 have exponent ≤ 5.

$$\rho_3^1 = 0.25 \quad \rho_3^0 = 1.00 \quad p_3 = 0.20$$

Step 3 — Summary and Expected Feasibility Rate

	Item 1	Item 2	Item 3
Weight	3	2	4
ρ_j^1	0.50	0.50	0.25
ρ_j^0	0.75	0.75	1.00
p_j **(ours)**	**0.40**	**0.40**	**0.20**
p_j (uniform)	0.50	0.50	0.50

Table 2. *Per-item feasibility densities and sampling probabilities for the single-constraint example.*

Item 3 consumes 80% of the capacity upon inclusion; the framework pushes p_3 to 0.20. Enumerating all 8 strings explicitly, the expected feasibility rate under our method is **87.2**% versus 62.5% under uniform initialization—a gain of 24.7 percentage points.

5.2 Multiple Constraint Example

Consider the following MKP instance with $n = 4$ items and $m = 2$ constraints:

	Item 1	Item 2	Item 3	Item 4
Weight (C1)	2	4	3	5
Weight (C2)	3	2	5	1
Value	4	7	5	3

Table 3. *4-item two-constraint instance. $W_1 = 7$, $W_2 = 8$.*

Of the $2^4 = 16$ possible solutions, 9 are feasible (overall density 56.25%). The two-constraint setting requires evaluating each completion against both thresholds simultaneously. We work through Item 4 in full to illustrate how the multi-dimensional threshold check works, then summarize all four items.

Sub-generating function for Item 4 ($w_{41} = 5$, $w_{42} = 1$):

$$g_4(x_1, x_2) = (1 + x_1^2 x_2^3)(1 + x_1^4 x_2^2)(1 + x_1^3 x_2^5)$$

When Item 4 is **included**, the remaining capacity is $W_1 - w_{41} = 2$ and $W_2 - w_{42} = 7$. When Item 4 is **excluded**, the full capacity $W_1 = 7$, $W_2 = 8$ is available. Table 4 enumerates all $2^3 = 8$ completions of g_4.

$\{1,2,3\}$ subset	e_1	e_2	Incl	Excl
$\emptyset$	0	0	✓	✓
$\{3\}$	3	5	✗	✓
$\{2\}$	4	2	✗	✓
$\{2,3\}$	7	7	✗	✓
$\{1\}$	2	3	✓	✓
$\{1,3\}$	5	8	✗	✓
$\{1,2\}$	6	5	✗	✓
$\{1,2,3\}$	9	10	✗	✗

Table 4. *All 8 completions of g_4. e_1, e_2 are total weights from items 1–3. Incl: $e_1 \leq 2$, $e_2 \leq 7$. Excl: $e_1 \leq 7$, $e_2 \leq 8$.*

From Table 4: 2 of 8 completions are feasible when Item 4 is included; 7 of 8 are feasible when it is excluded.

$$\rho_4^1 = 0.250 \quad \rho_4^0 = 0.875 \quad p_4 = 0.222$$

Item 4 consumes 71% of W_1 on its own, leaving almost no room for any other item on C1 when it is included—which is why p_4 is pushed down to 0.222. Notice that this signal comes specifically from C1: Item 4's weight on C2 is only $w_{42} = 1$, which is harmless. The two-dimensional threshold check automatically identifies which constraint is doing the work.

The same procedure applied to all four items yields:

	Item 1	Item 2	Item 3	Item 4
Weight (C1)	2	4	3	5
Weight (C2)	3	2	5	1
ρ_j^1	0.500	0.375	0.375	0.250
ρ_j^0	0.625	0.750	0.750	0.875
p_j **(ours)**	**0.444**	**0.333**	**0.333**	**0.222**
p_j (uniform)	0.500	0.500	0.500	0.500

Table 5. *Per-item probabilities for the two-constraint example.*

Enumerating all 16 strings with these p_j values, the expected feasibility rate under our method is **83.8**% versus 56.25% under uniform initialization—a gain of 27.6 percentage points.

Note that Items 2 and 3 are penalized equally ($p_j = 0.333$) despite very different weight profiles: Item 2 is heavy on C1 and light on C2, while Item 3 is the reverse. Both are constrained equally by the combination of the two thresholds. This joint accounting—automatically identifying which constraints bind for each item—is what distinguishes the multi-constraint GF approach from heuristics based on single constraints considered independently.

6 Experiments

6.1 Setup

We generate a synthetic MKP instance with $n = 100$ items and $m = 3$ constraints using a fixed random seed (42). Item weights w_{ji} are drawn uniformly from $[1, 100]$ and values v_j uniformly from $[1, 100]$. Capacities are set to $W_i = \alpha \cdot \sum_j w_{ji}$ for $\alpha \in \{0.25, 0.50, 0.75\}$, giving three instances ranging from tight to loose.

We compare two initialization strategies within an otherwise identical steady-state GA:

1. **Uniform (baseline):** Each bit set to 1 with probability 0.5 independently.

2. **GF-Biased:** Each bit j set to 1 with probability $\hat{p}_j$ estimated via Monte Carlo (Section 4.2).

We use a **death penalty** fitness model: infeasible solutions receive fitness 0 and are non-competing in selection. This isolates the effect of initialization cleanly, since no repair mechanism can compensate for an all-infeasible initial population.

For the extended experiments at $\alpha = 0.25$ (Sections 7.3–7.4), we additionally evaluate a hybrid initialization sweep—varying the fraction of GF-biased solutions in the initial population—followed by an ILS post-processing extension to assess the quality ceiling reachable with downstream optimization:

3. **Hybrid:** A mixed initial population combining GF-biased and uniformly initialized solutions at varying proportions (0%, 25%, 50%, 75%, 100% GF-biased), to identify the minimum informed fraction needed for reliable convergence.

4. **GF-Biased + ILS:** GF-biased initialization followed by iterated local search post-processing ($r = 50$ restarts, $k = 5$ bit-flip perturbations), to show the quality ceiling achievable with standard local search on top of the GF output.

6.2 Parameters

Parameter	Value
Population size P	50
Offspring per run	20,000
Mutation rate	$1/n = 0.01$
Crossover	Uniform
Selection	Binary tournament ($k = 2$)
Fitness	Death penalty
Independent runs	20
Monte Carlo samples M	10^7
ILS restarts r	50
ILS perturbation size k	5 (bit-flips)

Table 6. *Experimental parameters. ILS parameters apply only to Section 7.4.*

6.3 Reference Best

A reference best fitness is established per α value by running a repair-based GA (Chu & Beasley) for 30,000 offspring over 20 runs and taking the best observed value. These values (2375 at $\alpha = 0.25$; 3909 at $\alpha = 0.50$; 4823 at $\alpha = 0.75$) are not proven optimal but serve as a consistent performance ceiling for relative comparisons. All quality figures reported below are expressed as a percentage of this ceiling. We note that with ILS post-processing, the GF-biased approach reaches within 0.5 percentage points of this ceiling on the tight instance (Section 7.4).

7 Results

7.1 Sampling Probability Behavior

Table 7 summarizes the distribution of estimated $\hat{p}_j$ values across all items at each tightness level.

α	Min	Max	Mean	Interpretation
0.25	0.000	1.000	0.240	Strong diff. (tight)
0.50	0.414	0.493	0.449	Weak diff. (medium)
0.75	0.500	0.500	0.500	None (loose)

Table 7. *Distribution of $\hat{p}_j$ across all 100 items at each tightness level.*

At $\alpha = 0.75$, every item has $\hat{p}_j = 0.5$ by construction: the instance is loose enough that including or excluding any single item has negligible effect on feasibility, so the framework degenerates automatically to uniform initialization. This is a theoretical prediction of the framework (Section 3.4), and Table 7 confirms it holds exactly. The identical results at $\alpha = 0.75$ reported below are therefore not a comparison between two methods—they are a validation that the self-calibrating property holds empirically. At $\alpha = 0.25$, the range $[0, 1]$ indicates that some items are so weight-dominant that they are never compatible with any feasible completion ($p_j = 0$), while others are always compatible ($p_j = 1$).

7.2 Initialization Feasibility and Final Fitness

Table 8 reports initialization feasibility and final solution quality jointly. We present them together because at $\alpha = 0.25$ the two are causally linked in the most direct sense possible: zero initial feasibility under uniform initialization means selection never operates, so final fitness is also zero. The table tells a single story about what the initialization method enables.

α	Method	Init Feas.	Final (% ref)
0.25	Uniform	0.0%	0.0%
0.25	GF-Biased	**52.5%**	**96.5%**
0.50	Uniform	32.6%	98.5%
0.50	GF-Biased	**71.6%**	**98.8%**
0.75	Uniform	100.0%	99.5%
0.75	GF-Biased	100.0%	99.5%

Table 8. *Initialization feasibility and final fitness under a death penalty model. Averages over 20 runs. The $\alpha = 0.75$ rows are identical because $p_j = 0.5$ for all items at this tightness level.*

The result at $\alpha = 0.25$ is the central finding. Under uniform initialization, every solution in every initial population across all 20 runs is infeasible. The death penalty assigns fitness 0 to all solutions, selection cannot differentiate between them, and no improvement ever occurs. GF-biased initialization starts with 52.5% of the initial population feasible, selection operates immediately, and the GA converges to 96.5% of the reference best. This is not a marginal improvement—it is the difference between a completely inoperative algorithm and one that solves the problem effectively.

At $\alpha = 0.50$, both methods eventually converge to comparable final quality (98.5% versus 98.8%), but GF-biased initialization starts with over twice the fraction of feasible solutions (71.6% versus 32.6%) and converges faster (Table 11).

7.3 Hybrid Initialization

Before adding post-processing, we first examine how much of the population needs to be GF-biased for the algorithm to function at all. Table 9 sweeps the informed fraction at $\alpha = 0.25$.

Inf. Frac.	Init Feas.	Quality (% ref)	Solved
0%	0.0%	0.0%	0/20
25%	11.6%	97.1%	20/20
50%	25.1%	96.7%	20/20
75%	37.9%	97.1%	20/20
100%	50.5%	96.6%	18/20

Table 9. *Hybrid initialization sweep at $\alpha = 0.25$. "Inf. Frac." is the GF-biased proportion of the initial population; the remainder are uniform. "Solved" counts runs reaching $\geq 95\%$ of the reference best. 20 runs per condition.*

Seeding just 25% of the population with GF-biased solutions—while initializing the remaining 75% uniformly—is sufficient to rescue the algorithm: all 20 runs converge to 97%+ quality, compared to 0/20 under pure uniform initialization. Mean solution quality is approximately flat across all non-zero informed fractions (96.6–97.1%), suggesting that the critical transition is binary: any non-zero informed fraction crosses the feasibility threshold required for selection to operate. Notably, the pure GF condition (100% informed) solves 18/20 runs rather than 20/20, while mixed conditions at 25–75% solve all 20; this is consistent with lower population diversity at initialization when all solutions are drawn from the same biased distribution. The informed fraction is itself a tunable parameter; practitioners can trade off initialization overhead against early-generation diversity as their application demands.

7.4 Full Pipeline at $\alpha = 0.25$

Table 10 shows what is achievable at $\alpha = 0.25$ as downstream optimization is added to the GF-biased baseline.

Method	Quality (% ref)	Solved
Uniform + death penalty	0.0%	0/20
GF-Biased	96.5%	20/20
GF-Biased + ILS ($r = 50$, $k = 5$)	**99.3%**	20/20

Table 10. *Solution quality at $\alpha = 0.25$ across pipeline configurations. 20 runs each. "Solved" counts runs reaching $\geq 95\%$ of the reference best. A repair-based GA achieves 99.8% as a non-comparable reference (it uses objective values unavailable to the death-penalty pipeline).*

Adding a lightweight ILS step to the GF-biased GA brings mean quality to 99.3% of the reference best. The ILS configuration used here ($r = 50$ restarts, $k = 5$ bit-flip perturbations) is one point in a tunable parameter space; we do not claim these settings are optimal. The point is that the output of the GF framework slots directly into standard local search techniques, and substantial quality gains are available through straightforward downstream optimization.

7.5 Convergence Speed

Table 11 reports the median offspring evaluated to reach each fitness threshold at $\alpha \in \{0.25, 0.50\}$. The $\alpha = 0.75$ case is omitted as both methods are identical by construction. Convergence speed is not a primary contribution of this work; these results are included for completeness.

α	Method	70%	80%	90%	95%	98%
0.25	Uniform	never (0/20 runs)				
0.25	GF	481	1,023	2,309	7,529	16,845*
0.50	Uniform	82	316	893	1,740	8,327
0.50	GF	54	255	788	1,577	6,724

Table 11. *Median offspring to first reach each fitness threshold (% of reference best). *reached in only 1 of 20 runs within 20,000 offspring.*

GF-biased initialization also yields modestly faster early convergence at $\alpha = 0.50$, consistent with the higher initial feasibility giving selection an earlier head start.

8 Discussion

8.1 When Does GF-Biased Initialization Help?

The results suggest a clear characterization of when the framework adds value. It helps when (1) the instance is tight enough that uniform initialization produces non-trivial infeasibility ($\alpha \lesssim 0.50$), and (2) the fitness model is constraint-respecting without repair—whether that is a death penalty, a soft penalty with a high coefficient, or the absence of an objective function entirely. When the instance is loose ($\alpha = 0.75$), the framework degenerates to uniform initialization automatically. This graceful degradation is a theoretical property of the p_j formula, not a tuned behavior, and the experiments confirm it.

8.2 Applicability Beyond Standard MKP

The framework's most important property is that it is *objective-free*: p_j depends only on weights w_{ji} and capacities W_i, not on values v_j. This makes it applicable to settings where repair operators are undefined or unreliable:

- **Constraint satisfaction problems (CSPs):** When no objective function exists, the efficiency ranking u_j cannot be computed and repair is undefined. GF-biased initialization provides a principled starting point for any constraint-feasible-only search.

- **Noisy or uncertain objectives:** When value estimates are unreliable, u_j may be misleading. The GF framework ignores objective values entirely and remains well-calibrated.

- **Adversarial correlation structure:** Hill and colleagues [6] showed that standard greedy repair degrades on instances where item weights and values are negatively correlated, because the efficiency ranking systematically disadvantages heavy, high-value items. GF-biased initialization is unaffected by value-weight correlation. Demonstrating this advantage empirically is a primary direction for follow-up work (Section 9.1).

8.3 Computational Overhead

The one-time precomputation at $M = 10^7$, $n = 100$, $m = 3$ runs in 5–8 seconds on a T4 GPU (Section 4.3). For larger-scale applications (e.g., $n = 500$ as in OR-Library benchmarks), the cost scales as $O(n \cdot M \cdot m)$—linearly in n—and is fully GPU-parallelizable across the sample dimension.

Amortization across population size. The precomputation cost is fixed per instance, independent of population size P and offspring budget. The amortized cost per initialized solution is therefore T_{precomp}/P: approximately 0.10–0.16 seconds per solution at $P = 50$, dropping to 0.025–0.04 seconds at $P = 200$. Larger populations make the precomputation relatively cheaper per solution—the opposite of the repair operator, whose cost is incurred once per offspring regardless of population size.

Amortization across runs. If the same instance is evaluated k times—for hyperparameter tuning, multi-seed experiments, or benchmark evaluation—the p_j vector is computed once and reused across all k runs. The amortized precomputation cost per run is T_{precomp}/k, which vanishes as k grows. This is directly relevant to standard benchmark practice: OR-Library evaluations typically run 20–30 trials per instance, reducing the effective precomputation overhead to well under one second per trial.

Contrast with repair. These cost structures differ fundamentally. The repair operator is applied to every infeasible offspring; at $\alpha = 0.25$ under uniform initialization, essentially every offspring is infeasible, so repair runs on close to 100% of evaluations throughout the GA budget. Its total cost therefore scales linearly with offspring count T and problem size n. GF precomputation does not scale with T at all: regardless of how many offspring are evaluated, the p_j values are computed exactly once.

8.4 Limitations

- **Order-1 approximation:** The framework estimates per-item feasibility independently using order-1 schemas. For items with strong pairwise interactions, the order-1 approximation may over- or under-estimate the true per-item feasibility contribution. Extending to order-2 schemas is straightforward but increases computation by a factor of n.

- **Single-instance generalization:** Our experiments use a single instance per tightness level. Results on OR-Library benchmark instances will be presented in follow-up work.

- **Interaction with repair:** On well-structured standard MKP instances, repair effectively compensates for poor initialization and GF-biased initialization offers little advantage. The benefit is expected to emerge on adversarial or objective-free instances where repair is unavailable.

- **ILS and hybrid parameters:** The ILS configuration ($r = 50$, $k = 5$) and the hybrid informed fraction are unoptimized. The experiments show that these extensions can substantially improve quality; they do not establish optimal settings.

9 Conclusion and Future Work

We have presented a generating function-informed initialization framework for genetic algorithms on the Multidimensional Knapsack Problem. The framework draws on the same algebraic tradition—encoding combinatorial structure as coefficients of formal power series—that Hardy and Ramanujan used to analyze integer partitions, and applies it to a new question: how much feasible space does each item's inclusion or exclusion preserve? By computing per-item feasibility densities from sub-generating functions and deriving per-bit sampling probabilities proportional to these densities, we obtain an initialization method that is principled, purely constraint-driven, and self-calibrating: it degenerates to uniform initialization exactly when no constraint structure is present.

The central empirical result is that on tight instances ($\alpha = 0.25$) under a death penalty fitness model, GF-biased initialization is the only method that functions at all—producing 52.5% initial feasibility and converging to 96.5% of the reference best, while uniform initialization finds no feasible solution in 20 independent runs. Adding lightweight ILS post-processing raises this to 99.3%. Seeding as little as 25% of the population with GF-biased solutions is sufficient to rescue the algorithm entirely. On medium instances ($\alpha = 0.50$), GF-biased initialization converges faster while matching final quality. On loose instances ($\alpha = 0.75$), both methods are

identical by theoretical prediction.

9.1 Future Work

1. **Adversarial instances (Part 2):** Generating MKP instances following the protocol of Hill et al. [6] with negative value-weight correlation to demonstrate regimes where repair-based approaches degrade while GF-biased initialization remains effective.
2. **Constraint satisfaction:** Applying the framework to pure CSP instances where repair operators are undefined.
3. **OR-Library benchmarks:** Scaling to standard benchmark instances with n up to 500 and m up to 30. The standard practice of running 20–30 trials per instance makes the amortized precomputation cost per trial negligible.
4. **Higher-order schemas:** Extending feasibility density estimation to order-2 schemas to capture pairwise item interactions.
5. **Integration with repair:** Investigating whether GF-biased initialization improves the starting point for repair-based GAs on tight benchmark instances.

References

[1] P. C. Chu and J. E. Beasley, *A Genetic Algorithm for the Multidimensional Knapsack Problem*, Journal of Heuristics, 4(1):63–86, 1998.

[2] J. E. Beasley, *OR-Library: Distributing Test Problems by Electronic Mail*, Journal of the Operational Research Society, 41(11):1069–1072, 1990.

[3] J. H. Holland, *Adaptation in Natural and Artificial Systems*, University of Michigan Press, Ann Arbor, MI, 1975.

[4] H. S. Wilf, *generatingfunctionology*, 2nd edition, Academic Press, 1994.

[5] H. Kellerer, U. Pferschy, and D. Pisinger, *Knapsack Problems*, Springer, Berlin, 2004.

[6] R. R. Hill, Y. K. Cho, and J. T. Moore, *Problem Generator Algorithms for the Multidimensional Knapsack Problem*, In *Proceedings of the 45th Winter Simulation Conference*, 2013.

[7] G. H. Hardy and S. Ramanujan, *Asymptotic Formulae in Combinatory Analysis*, Proceedings of the London Mathematical Society, 2(17):75–115, 1918.